ЧАСТЬ 1. ОСНОВЫ PYTHON • ТЕМА 1.5

1.5. Циклы

Практика. Занятие 1 из 2: while, for и range, break и continue

К концу занятия ты сможешь:

- повторять действия циклом while и вовремя его останавливать
- считать с нужного числа, до нужного числа и с нужным шагом через for и range
- перебирать строку по буквам
- выходить из цикла через break и пропускать шаг через continue

Вспомним: ввод числа (тема 1.2)

input() всегда возвращает строку. int() превращает строку в целое число, и с ним уже можно считать.

main.py

```
age = int(input("Сколько тебе лет? "))
print(f"Через 5 лет тебе будет {age + 5}")
```

Вывод в консоли

```
Сколько тебе лет? 14 <- ввод
Через 5 лет тебе будет 19
```

Стрелка «<- ввод» — пометка для тебя, в программе её нет.

Без int() значение останется строкой, и прибавить к нему число не получится.

Вспомним: if, elif, else (тема 1.4)

Python проверяет условия сверху вниз и выполняет первую подходящую ветку. else — если не подошло ничего.

main.py

```
score = int(input("Очки за матч: "))
if score >= 100:
    print("Новый рекорд")
elif score >= 50:
    print("Хорошая игра")
else:
    print("Попробуй ещё раз")
```

Запуск 1

```
Очки за матч: 75 <- ввод
Хорошая игра
```

Запуск 2

```
Очки за матч: 30 <- ввод
Попробуй ещё раз
```

Вспомним: сравнения, and и or (тема 1.4)

and — True, когда верны обе части. or — True, когда верна хотя бы одна.

main.py

```
level = 3
coins = 120
can_buy = level >= 5 and coins >= 100
print(f"Можно в магазин: {can_buy}")
is_weekend = False
is_holiday = True
print(f"Можно отдыхать: {is_weekend or is_holiday}")
```

Вывод в консоли

```
Можно в магазин: False
Можно отдыхать: True
```

Сегодня это пригодится для проверки «число вне диапазона»: guess < 1 or guess > 100.

План практики

№	Задание	Что понадобится	Уровень	Урок	Минуты
1	Обратный отсчёт до старта	while, счётчик	1	1	8
2	Зависшая ракета	бесконечный цикл, Ctrl+C, while	2	1	10
3	План тренировки	for, range(start, stop, step)	3	1	12
4	Статистика ника	for по строке, if, счётчик	4	2	10
5	Копилка на мечту	while True, break, накопитель	5	2	15
6	Угадай число	while True, break, continue, or	6	2	15
—	Мини-проект «Дневник тренировок»	всё, что прошли в этом занятии	—	3	25–30
—	Итоги	—	—	3	5–10

Уровень 1 — самое простое задание, дальше сложнее. ★★ «Для сильных» — после заданий 3, 5 и 6, если основное уже готово.

Задание 1. Обратный отсчёт до старта

Программа спрашивает, с какого числа начать отсчёт.

Затем выводит числа по одному в строке: от этого числа вниз до 1.

В конце один раз выводит «Старт!».

Что понадобится: while и счётчик — 1.5; -= — 1.3; input() и int() — 1.2; print — 0.2

Подсказка: условие цикла — «пока счётчик больше нуля». Внутри цикла выведи счётчик и уменьши его на 1. «Старт!» пишется уже после цикла, без отступа.

★ **Успел раньше:** если ввели 0 или отрицательное число, программа пишет «Отсчитывать нечего» вместо отсчёта.

Что должно получиться

Вывод в консоли

```
С какого числа считать? 5 <- ввод
5
4
3
2
1
Старт!
```

Стрелка «<- ввод» — пометка для тебя, в программе её нет.

Проверь себя

- При вводе 5 выводится ровно пять чисел, последнее — 1.
- «Старт!» печатается один раз, после всех чисел.
- Ноль на экране не появляется.

Задание 2. Зависшая ракета

Перепиши программу и запусти. Строка будет выводиться снова и снова: программа зависла.

Останови её, прочитай traceback и найди, почему цикл не заканчивается. Почини: топливо должно убывать до нуля, а в конце — «Ракета на орбите».

rocket.py

```
fuel = 3
while fuel > 0:
    print(f"Топливо: {fuel}. Двигатель работает")
print("Ракета на орбите")
```

Что понадобится: while — 1.5; бесконечный цикл и Ctrl+C — 1.5; -= — 1.3

Как остановить зависшую программу

Терминал VS Code: щёлкни в терминале и нажми Ctrl+C.

Thonny: красная кнопка «Стоп».

Подсказка: traceback — это сообщение об ошибке. Читай его снизу вверх. Спроси себя: какая переменная должна меняться внутри цикла?

★ **Успел раньше:** топливо вводит пользователь, а в конце программа пишет, сколько шагов проработал двигатель.

Что должно получиться: «Зависает» и «Работает»

Зависает: до исправления

Вывод в консоли

```
Топливо: 3. Двигатель работает
Топливо: 3. Двигатель работает
Топливо: 3. Двигатель работает
Топливо: 3. Двигатель работает
```

… и так без конца, пока не нажмёшь Ctrl+C. Потом:

```
Traceback (most recent call last):
  File "/home/claude/demo/rocket.py", line 3, in <module>
    print(f"Топливо: {fuel}. Двигатель работает")
    ~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
KeyboardInterrupt
```

Путь к файлу у тебя будет свой. Снизу вверх: KeyboardInterrupt — программу прервали вручную; выше — строка 3, на которой крутился цикл.

Работает: после исправления

Вывод в консоли

```
Топливо: 3. Двигатель работает
Топливо: 2. Двигатель работает
Топливо: 1. Двигатель работает
Ракета на орбите
```

Проверь себя

- Программа останавливается сама, без Ctrl+C.
- Топливо идёт 3, 2, 1, а 0 уже не выводится.
- Ты можешь объяснить, из-за какой строки цикл не кончался.

Задание 3. План тренировки

Программа спрашивает, сколько будет упражнений и сколько минут длится тренировка. Потом выводит три строки:

- номера упражнений от 1 до введённого числа;
- отметки минут от 0 до конца тренировки с шагом 5;
- отсчёт отдыха от 10 до 1.

В конце — «Тренировка закончена».

Ловушка: range не включает stop

main.py

```
for number in range(1, 10):
    print(number, end=" ")
print()
```

Вывод в консоли

```
1 2 3 4 5 6 7 8 9
```

Что понадобится: for и range(start, stop, step) — 1.5; print(..., end=" ") — 1.2; input() и int() — 1.2

Подсказка: чтобы дойти до нужного числа, stop делай на 1 больше. Для счёта вниз шаг отрицательный. Пустой print() переводит на новую строку.

★ **Успел раньше:** шаг отметок минут тоже вводит пользователь.

Что должно получиться

Вывод в консоли

```
Сколько упражнений? 8 <- ввод
Сколько минут тренировка? 30 <- ввод
Упражнения: 1 2 3 4 5 6 7 8
Отметки минут: 0 5 10 15 20 25 30
Отдых: 10 9 8 7 6 5 4 3 2 1
Тренировка закончена
```

Стрелка «<- ввод» — пометка для тебя, в программе её нет.

Проверь себя

- Последнее упражнение — 8, а не 7.
- Отметки минут заканчиваются ровно на 30.
- Отдых идёт от 10 до 1, нуля нет.

★★ Счастливые числа

Найди все трёхзначные числа, у которых произведение цифр равно сумме цифр.

Программа выводит каждое такое число в отдельной строке, а в конце — сколько их всего.

Цифры числа ты уже умеешь получать через // и % (тема 1.3).

Пример результата (начало и конец)

```
123
…
Найдено: …
```

Первое такое число — 123: 1 × 2 × 3 = 6 и 1 + 2 + 3 = 6. Сколько их всего, узнай сам.

Задание 4. Статистика ника

Программа просит ввести ник для игры.

Выводит каждую букву ника с её номером, по одной в строке: «1 - P».

В конце пишет длину ника и сколько в нём маленьких латинских букв a.

Что понадобится: for по строке — 1.5; счётчик — 1.5; if и == — 1.4; f-строки — 1.2

Подсказка: функцию len() мы ещё не проходили, поэтому длину считай сам: счётчик, который растёт на 1 на каждой букве. Номер буквы и длина — это один и тот же счётчик.

★ **Успел раньше:** считай и маленькую a, и заглавную A.

Что должно получиться

Вывод в консоли

```
Введи ник: PandaGo <- ввод
1 - P
2 - a
3 - n
4 - d
5 - a
6 - G
7 - o
Длина ника: 7
Букв a: 2
```

Стрелка «<- ввод» — пометка для тебя, в программе её нет.

Проверь себя

- Номера идут с 1, а не с 0.
- Для PandaGo программа насчитала 2 буквы a.
- Длина ника совпадает с номером последней буквы.
- Итоговые строки выводятся один раз, после всех букв.

Задание 5. Копилка на мечту

Программа спрашивает, сколько стоит мечта: наушники, игра, кроссовки.

Потом снова и снова спрашивает, сколько ты отложил, и после каждой суммы пишет, сколько в копилке.

Ввод 0 — конец. Программа пишет, сколько накоплено, и либо «Цель достигнута, можно покупать», либо сколько осталось накопить.

Что понадобится: while True и break — 1.5; накопитель — 1.5; += — 1.3; if/else — 1.4; input() и int() — 1.2

Подсказка: while True повторяет без конца, поэтому выход нужен внутри цикла. Проверку на 0 ставь сразу после ввода, до прибавления. Накопитель создают до цикла со значением 0.

★ **Успел раньше:** отрицательную сумму не прибавлять, а писать «Сумма не может быть меньше нуля».

Что должно получиться

Запуск 1: цель не достигнута

```
Сколько стоит мечта? 3000 <- ввод
Сколько отложил? (0 - закончить) 500 <- ввод
В копилке: 500
Сколько отложил? (0 - закончить) 700 <- ввод
В копилке: 1200
Сколько отложил? (0 - закончить) 300 <- ввод
В копилке: 1500
Сколько отложил? (0 - закончить) 0 <- ввод
Всего накоплено: 1500
Осталось накопить: 1500
```

Запуск 2: цель достигнута

```
Сколько стоит мечта? 1000 <- ввод
Сколько отложил? (0 - закончить) 600 <- ввод
В копилке: 600
Сколько отложил? (0 - закончить) 500 <- ввод
В копилке: 1100
Сколько отложил? (0 - закончить) 0 <- ввод
Всего накоплено: 1100
Цель достигнута, можно покупать
```

Проверь себя

- После ввода 0 строки «В копилке» нет.
- Итог печатается один раз, после цикла.

Стрелка «<- ввод» — пометка для тебя, в программе её нет.

★★ Шаги Коллатца

Пользователь вводит целое число больше нуля. Пока число не стало 1, программа меняет его:

- чётное — делит на 2;
- нечётное — умножает на 3 и прибавляет 1.

Программа выводит всю цепочку в одну строку и сколько понадобилось шагов.

Пример результата

```
Введи число: 6 <- ввод
6 3 10 5 16 8 4 2 1
Шагов: 8
```

Попробуй 27: шагов будет больше сотни, а числа по пути вырастут до нескольких тысяч. Почему цепочка всегда приходит к 1, до сих пор никто не доказал.

Задание 6. Угадай число

Число задано в коде: SECRET = 42. Игрок вводит числа, программа отвечает «Загаданное больше» или «Загаданное меньше».

Число вне диапазона 1–100 — не попытка: программа пишет «Нужно число от 1 до 100» и сразу просит снова.

Когда игрок угадал, программа пишет, сколько было попыток, и заканчивает работу.

Что понадобится: while True, break, continue — 1.5; счётчик — 1.5; if/elif/else, or и сравнения — 1.4

Подсказка: порядок в цикле важен: ввод → проверка диапазона → счётчик попыток → сравнение с SECRET. Если счётчик стоит раньше проверки, ошибочный ввод тоже станет попыткой. Застрял — 15 минут ищи сам, потом спроси: что делал, что ожидал, что получил.

★ **Успел раньше:** не больше 7 попыток; после седьмой неудачной — «Попытки закончились, было загадано 42».

Что должно получиться

Вывод в консоли

```
Твоё число (1-100): 50 <- ввод
Загаданное меньше
Твоё число (1-100): 150 <- ввод
Нужно число от 1 до 100
Твоё число (1-100): 25 <- ввод
Загаданное больше
Твоё число (1-100): 37 <- ввод
Загаданное больше
Твоё число (1-100): 0 <- ввод
Нужно число от 1 до 100
Твоё число (1-100): 42 <- ввод
Угадал. Попыток: 4
```

Стрелка «<- ввод» — пометка для тебя, в программе её нет.

Проверь себя

- Ввели 6 чисел, но попыток 4: 150 и 0 не считаются.
- После «Угадал» программа больше ничего не спрашивает.
- На 1 и на 100 программа отвечает «больше» или «меньше», а не про диапазон.

★★ Умный угадыватель

Теперь наоборот: число от 1 до 100 загадывает человек, а угадывает программа.

Программа называет число, человек отвечает «больше», «меньше» или «угадал».

Цель — угадать любое число не больше чем за 7 попыток.
Придумай, какое число называть каждый раз.

Подумай: какие две переменные описывают отрезок, где число ещё может быть?

Пример: загадано 70

```
Это 50? больше <- ввод
Это 75? меньше <- ввод
Это 62? больше <- ввод
Это 68? больше <- ввод
Это 71? меньше <- ввод
Это 69? больше <- ввод
Это 70? угадал <- ввод
Угадал за 7 попыток
```

Мини-проект. Дневник тренировок недели

Программа по очереди спрашивает, сколько минут ты тренировался в каждый день недели: футбол, бег, танцы — что угодно.

0 — день отдыха, –1 — закончить неделю раньше.

В конце — неделя одной строкой, сумма минут, число активных дней и оценка недели.

Это пригодится в Проекте 5 «Личный трекер»: там дневник тренировок будет запоминать данные между запусками.

Вывод в консоли

```
П: минут тренировки (0 - отдых, -1 - стоп): 40 <- ввод
В: минут тренировки (0 - отдых, -1 - стоп): 0 <- ввод
С: минут тренировки (0 - отдых, -1 - стоп): 30 <- ввод
Ч: минут тренировки (0 - отдых, -1 - стоп): 45 <- ввод
П: минут тренировки (0 - отдых, -1 - стоп): 0 <- ввод
С: минут тренировки (0 - отдых, -1 - стоп): 60 <- ввод
В: минут тренировки (0 - отдых, -1 - стоп): 20 <- ввод
Неделя: П _ С Ч _ С В
Всего минут: 195
Активных дней: 5
Отличная неделя
```

Стрелка «<- ввод» — пометка для тебя, в программе её нет.

Что умеет программа

1. Спрашивает минуты по каждому дню: П, В, С, Ч, П, С, В.
2. Не считает дни отдыха, но отмечает их в строке недели знаком _.
3. По –1 заканчивает неделю раньше и сразу подводит итог.
4. Пишет сумму минут, число активных дней и оценку недели.

Запуск 2: неделя закончилась раньше

```
П: минут тренировки (0 - отдых, -1 - стоп): 20 <- ввод
В: минут тренировки (0 - отдых, -1 - стоп): 0 <- ввод
С: минут тренировки (0 - отдых, -1 - стоп): 25 <- ввод
Ч: минут тренировки (0 - отдых, -1 - стоп): -1 <- ввод
Неделя: П _ С
Всего минут: 45
Активных дней: 2
Добавь движения
```

Оценка: от 150 минут — «Отличная неделя», от 60 — «Хорошее начало», меньше — «Добавь движения».

План по шагам

Шаг 1. Перебери дни

Цикл for по строке DAYS. Внутри спроси минуты через int(input(...)), в вопросе — буква дня.

Темы: 1.5, 1.2

Проверка: запусти — программа задаёт 7 вопросов, первый начинается с «П:».

Шаг 2. Накопи итог

Прибавляй минуты к сумме, считай активные дни, добавляй букву дня в строку недели. На 0 — добавь «_ » и continue.

Темы: 1.5, 1.3, 1.1

Проверка: временно выведи сумму после цикла. Для 40, 0, 30 и дальше нулей — 70.

Шаг 3. Ранний выход

Сразу после ввода: если –1 — break. Эта проверка стоит выше проверки на 0.

Тема: 1.5

Проверка: введи 20, 0, 25, –1 — больше вопросов нет.

Шаг 4. Итог и оценка

После цикла выведи строку недели, сумму и активные дни, затем оценку через if/elif/else.

Темы: 1.2, 1.4

Проверка: 40, 0, 30, 45, 0, 60, 20 — 195 минут и «Отличная неделя».

Заготовка

Перепиши заготовку и замени каждый комментарий своим кодом. Каждую строку ты должен уметь объяснить.

training_diary.py

```
DAYS = "ПВСЧПСВ"
total_minutes = 0
active_days = 0
week_line = "" # сюда добавляем буквы дней и «_ »

for day in DAYS:
    # спроси минуты: f"{day}: минут тренировки (0 - отдых, -1 - стоп): "
    # если -1 — выйди из цикла
    # если 0 — добавь "_ " в week_line и перейди к следующему дню
    # иначе — прибавь минуты, посчитай активный день, добавь букву дня

# выведи week_line, сумму минут и число активных дней
# выведи оценку недели через if / elif / else
```

Готово, если… и усложнения

Готово, если…

- программа задаёт вопрос по каждому из 7 дней;
- при вводе 0 день не попадает в сумму, а в строке недели стоит _;
- при вводе –1 вопросов больше нет, сразу выводится итог;
- для 40, 0, 30, 45, 0, 60, 20 выводится 195 минут, 5 активных дней и «Отличная неделя».

★ Среднее за тренировку

Выведи среднее за активный день. Если активных дней нет, вместо деления пиши «Тренировок не было».

```
Среднее за тренировку: 39.0 мин
```

★★ Цель и лучший день

В начале спроси цель на неделю в минутах. В конце напиши, сколько не хватило или на сколько перевыполнил, и назови лучший день.

```
Цель 180: перевыполнил на 15 минут
Лучший день: С (60 мин)
```

Если всё готово: проверка пароля

Программа просит придумать пароль и считает в нём длину, цифры и заглавные латинские буквы.

Надёжный пароль — от 8 символов, хотя бы 2 цифры и хотя бы 1 заглавная буква. Иначе программа пишет «Слабый пароль, попробуй ещё».

Символы можно сравнивать: "0" <= symbol <= "9" верно только для цифр. Так же проверяются буквы от "A" до "Z".

Дальше сам: подскажи пользователю, чего именно не хватает.

Запуск 1

```
Придумай пароль: kitty12 <- ввод
Длина: 7, цифр: 2, заглавных: 0
Слабый пароль, попробуй ещё
```

Запуск 2

```
Придумай пароль: SuperKitty2024 <- ввод
Длина: 14, цифр: 4, заглавных: 2
Надёжный пароль
```

Итоги. Теперь я умею…

- повторять действия циклом while, пока условие верно, и вести счётчик и накопитель;
- узнавать бесконечный цикл и останавливать его: Ctrl+C в терминале или «Стоп» в Thonny;
- считать с нужного числа, до нужного числа и с нужным шагом через for и range;
- перебирать строку по буквам;
- выходить из цикла через break и пропускать шаг через continue.

В следующий раз: вложенные циклы, случайные числа через random и else у цикла.